



AKSUM UNIVERSITY

FACALTY OF COMPUTING TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE
GROUP ASSIGNMENT OF DESIGN AND ANALYSIS OF ALORITHMS

TITLE: TERAVELING SALEMAN PROBLEMS

PREPARED BY: THRID YEAR COMPUTER SCIENCE STUDENTS

No	First Name	Last Name	ID Number
1	Dagmawit	Mersa	1603897
2	Samuel	Ataklti	1603833
3	Betellehem	Haile	1603359

SUMMITTED TO INSTRUCTOR: EKUBZGI
SUMMSION DATE: 29/2026 GC

Table of Contents

1. INTRODUCTION	1
2. PROBLEM STATEMENT	1
Objectives	1
Assumptions.....	1
3. DATASET DESCRIPTION	2
3.1 City Importance.....	2
3.2 Distance Matrix	2
4. PROBLEM FORMULATION	2
4.1.Objective Function	3
Constraints	3
5. ALGORITHM DESIGN	3
5.1 Greedy Nearest Neighbor Algorithm	3
Steps:.....	3
Advantages:.....	3
Disadvantages:	3
5.2 Branch and Bound Algorithm.....	3
Key Concepts:	3
Pruning Rules:.....	3
Advantages:.....	4
Disadvantages:	4
5.3 Optimization Strategy	4
6. IMPLEMENTATION	4
Project Structure:	4
Execution:	4
Key Concepts:	4
7. EXPERIMENTS AND RESULTS	4
Test 1: Greedy from Addis Ababa	4
TEST 2 : Greedy from Random City	5
Test 3: Multi-Start Greedy	5
Test 4: Branch and Bound.....	6
8. COMPLEXITY ANALYSIS.....	6
Greedy:.....	6
Branch and Bound:	6

9. REAL-WORLD APPLICATIONS.....	6
11. CHALLENGES	7
11.1 Computational Complexity	7
11.2 Lower Bound Design	7
11.3 Data Collection	7
12. FUTURE WORK	7

1. INTRODUCTION

The Traveling Salesman Problem (TSP) is one of the most widely studied problems in the field of combinatorial optimization and theoretical computer science. It belongs to the class of NP-hard problems, meaning that no known algorithm can solve all instances of the problem efficiently in polynomial time as the number of cities increases.

The importance of the TSP lies in the fact that although it is easy to define, it is extremely difficult to solve optimally. The number of possible routes grows factorially with the number of cities, making brute-force solutions impractical for large datasets.

Formally, the problem is defined as follows:

Given a set of cities and the distances between each pair of cities, find the shortest possible route that visits each city exactly once and returns to the starting city.

This problem has important real-world applications in logistics, transportation, manufacturing, and network optimization. Delivery companies, airline routing systems, and supply chain networks all rely on TSP-based models to reduce cost and improve efficiency.

In this project, the TSP is applied to real Ethiopian cities. A delivery truck starts from Addis Ababa and visits 14 other cities before returning to the starting point. The goal is to compute both an optimal and an approximate solution.

- Two approaches are compared:
- Greedy Nearest Neighbor algorithm (fast approximation)
- Branch and Bound algorithm (exact but computationally expensive)

2. PROBLEM STATEMENT

This project considers a real-world transportation scenario in Ethiopia. A logistics truck starts from Addis Ababa, which serves as the central hub due to its strategic importance, and must visit 14 other cities exactly once before returning to the starting city.

Objectives

- To compute the shortest possible route minimizing total distance
- To design an efficient approximation algorithm
- To compare exact and heuristic approaches

Assumptions

- Distances are fixed and symmetric
- Road traffic and delays are ignored
- Each city is visited exactly once

- The graph is fully connected

3. DATASET DESCRIPTION

The dataset consists of 15 major Ethiopian cities:

Addis Ababa, Dire Dawa, Bahir Dar, Mekelle, Hawassa, Jimma, Gondar, Adama, Dessie, Jijiga, Shashamane, Arba Minch, Axum, Nekemte, and Harar.

These cities represent major economic, historical, and transportation centers across Ethiopia.

3.1 City Importance

- Addis Ababa: Capital and logistics hub
- Dire Dawa: Trade and railway center
- Mekelle: Northern administrative hub
- Bahir Dar: Tourism and agriculture
- Hawassa: Industrial development center

3.2 Distance Matrix

A 15×15 adjacency matrix represents road distances between cities.

Properties:

- Distance from a city to itself = 0
- Symmetric: $d(i, j) = d(j, i)$
- Fully connected graph

Distances were approximated using mapping tools and transportation references including the Ethiopian Roads Authority (ERA).

4. PROBLEM FORMULATION

The TSP is modeled using graph theory:

- Graph type: Complete weighted graph
- Nodes: Cities
- Edges: Roads between cities
- Weights: Distances

Let:

- n = number of cities
- $d(i, j)$ = distance between city i and j

4.1.Objective Function

Minimize total travel distance:

$$\text{Mini} = \sum_{i=1}^n d(c_i, c_{i+1}), \text{ where } c_{n+1} = c_1$$

Constraints

- Each city is visited exactly once
- The route returns to the starting city
- No repetition of cities

5. ALGORITHM DESIGN

5.1 Greedy Nearest Neighbor Algorithm

This heuristic selects the nearest unvisited city at each step.

Steps:

- Start from a city
- Move to nearest unvisited city
- Repeat until all cities are visited
- Return to starting city

Advantages:

- Fast execution
- Simple implementation

Disadvantages:

- Not optimal
- Depends on starting city

5.2 Branch and Bound Algorithm

This is an exact optimization method that reduces search space using pruning.

Key Concepts:

- Upper bound: best known solution
- Lower bound: estimated minimum cost

Pruning Rules:

A path is discarded if:

- Current cost $\geq$ best solution
- Lower bound $\geq$ best solution

Advantages:

- Guarantees optimal solution
- Reduces unnecessary search

Disadvantages:

- High computation cost
- Still exponential in worst case

5.3 Optimization Strategy

Multiple Greedy runs were executed from different starting cities. The best result was used as the initial upper bound for Branch and Bound, improving pruning efficiency and reducing computation time.

6. IMPLEMENTATION

The project is implemented in Python 3.8+.

Project Structure:

- cities.py → dataset
- utils.py → helper functions
- greedy.py → Greedy algorithm
- branch_and_bound.py → exact solver
- main.py → execution

Execution:

```
python main.py
```

Key Concepts:

- Distance matrix representation
- Recursive search
- Pruning optimization
- Visited node tracking

7. EXPERIMENTS AND RESULTS

Four main experiments were conducted after implementation.

Test 1: Greedy from Addis Ababa

- Distance: ~10,300 km
- Time: ~10–15 ms
- Result: Fast but non-optimal solution

```

TSP – Ethiopian Cities | Interactive Demo

Choose a mode:

[1] Quick Demo      – 5 cities, see ALL routes, instant result
[2] Custom Cities   – pick your own cities (4–10), run both algorithms
[3] Step-by-Step Trace – watch Greedy make decisions step by step
[4] Full 15-City Run – complete project output (instructor submission)
[0] Exit

Enter choice: █

```

TEST 2 : Greedy from Random City

- Distance: ~10,450 km
- Time: ~10–15 ms
- Observation: Starting point affects result

```

GREEDY NEAREST NEIGHBOR result:
Route : Addis Ababa → Adama → Shashamane → Hawassa → Arba Minch → Addis Ababa
Total : 1,058 km

BRANCH & BOUND result:
Route : Addis Ababa → Adama → Shashamane → Hawassa → Arba Minch → Addis Ababa
Total : 1,058 km

VERDICT

Brute-force optimal : 1,008 km
Greedy result       : 1,058 km X not optimal
Branch & Bound      : 1,058 km X not optimal

B&B nodes explored  : 10
B&B nodes pruned    : 16
(Brute force needed : 24 routes)

```

Test 3: Multi-Start Greedy

- Best distance: ~10,276 km
- Slightly higher execution time
- Improved approximation quality

```

— GREEDY RESULT —
Route : Addis Ababa → Adama → Shashamane → Hawassa → Arba Minch
→ Jimma → Addis Ababa
Total : 1,464 km

— B&B RESULT (for comparison) —
Route : Addis Ababa → Adama → Shashamane → Hawassa → Arba Minch
→ Jimma → Addis Ababa
Total : 1,464 km

Greedy found the optimal route.
B&B explored 31 nodes, pruned 60.

```

➤

Test 4: Branch and Bound

- Optimal distance: ~9,764 km
- Time: ~5 minutes
- Guaranteed optimal solution

8. COMPLEXITY ANALYSIS

Greedy:

- Time: $O(n^2)$
- Space: $O(n)$

Branch and Bound:

- Worst case: $O(n!)$
- Practical: reduced by pruning

For 15 cities:

$15! \approx 1.3$ trillion possible routes

9. REAL-WORLD APPLICATIONS

- Logistics and delivery systems
- Airline routing
- Emergency response
- Manufacturing optimization
- E-commerce delivery

11. CHALLENGES

11.1 Computational Complexity

One of the major challenges encountered in this project is the extremely high computational complexity of the Traveling Salesman Problem. The number of possible routes increases factorially with the number of cities, meaning that for a dataset of n cities, there are $(n-1)!(n-1)!(n-1)!$ possible permutations.

As a result, even a relatively small dataset of 15 cities leads to trillions of possible routes. This makes brute-force approaches completely impractical, as they require excessive time and computational resources. Therefore, advanced optimization techniques such as Greedy heuristics and Branch and Bound were necessary to reduce the search space and obtain feasible solutions within a reasonable time.

11.2 Lower Bound Design

Another important challenge was the design of an effective lower bound heuristic for the Branch and Bound algorithm. The lower bound plays a critical role in determining whether a partial solution should be explored further or pruned.

A poor lower bound can lead to inefficient pruning, resulting in unnecessary computations and increased execution time. On the other hand, a well-designed admissible heuristic must never overestimate the actual minimum cost; otherwise, it may incorrectly eliminate optimal solutions.

In this project, simplified distance estimates were used to approximate the remaining cost from unvisited cities. Designing this heuristic required balancing accuracy and computational efficiency to ensure that pruning was both safe and effective.

11.3 Data Collection

Data collection was another significant challenge in this project due to the lack of a single standardized dataset for road distances between Ethiopian cities. Real-world road distances vary depending on routes, infrastructure conditions, and geographic terrain.

To address this, distances were estimated using multiple sources, including digital mapping tools and transportation references. In particular, data was cross-checked using available infrastructure information from relevant authorities to improve reliability.

In cases where exact values were not available, reasonable approximations were made based on the most commonly used road routes. These values were then validated for consistency to ensure that the final distance matrix was logically correct and suitable for algorithm testing.

12. FUTURE WORK

- Christofides algorithm
- Genetic algorithms
- Machine learning optimization
- Map-based visualization
- Expansion to 50+ cities